Threat Model Report

Secure Voting System

STRIDE | CVSS v3.1 | Attack Tree Analysis | DFD

Prepared by: _____Fiza Maheen_____

Registeration: _____SP23-BCT-016_____

Course: _____SSDD_____

Threat Modeling for Online Secure Voting System

1. Data Flow Diagram (DFD)

The Data Flow Diagram below illustrates how data moves between the key entities of the Secure Voting System. It identifies all trust boundaries, data stores, and external entities relevant to the threat model.

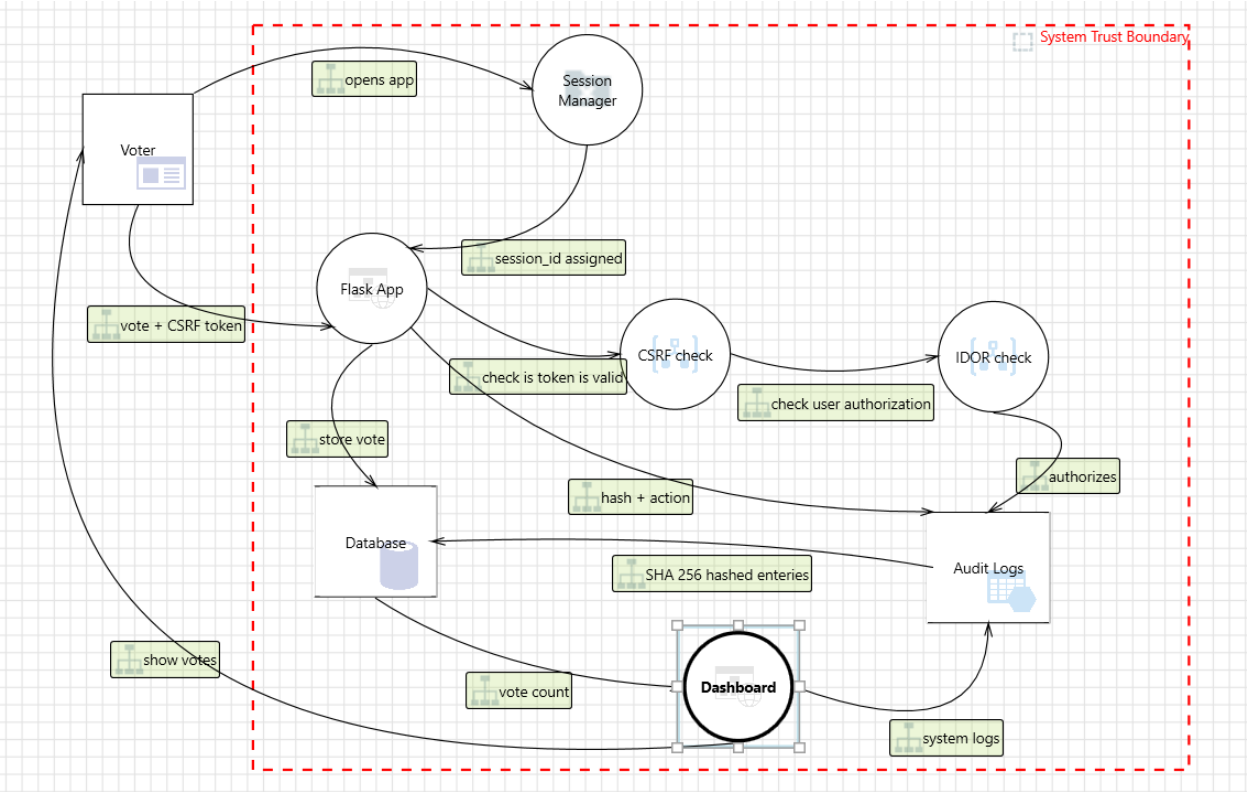
1.1 System Entities

- Voter (User)
- Flask App (Voting Application)
- Session Manager
- CSRF Check
- IDOR Check
- Database (stores votes)
- Audit Logs
- Dashboard (Results / Vote Count)

1.2 Data Flows

- Voter opens app → Session Manager (session_id assigned)
- Voter submits (vote + CSRF token) → Flask App
- Flask App performs (CSRF check) and (IDOR check)
- Valid vote is stored in → Database
- Flask App sends (hash + action) → Audit Logs (SHA-256 hashed entries)
- Dashboard fetches (vote count) and (show votes) from Database
- All actions are logged as → system logs

1.3 DFD Diagram using TMT



2. STRIDE Analysis

The STRIDE model identifies six threat categories applicable to the Secure Voting System. Each threat is mapped to a concrete attack scenario and a corresponding mitigation strategy.

STRIDE Category	Example in Voting System	Mitigation Strategy	Severity
Spoofing	Session hijacking or fake voter registration	Strong session management + MFA + proper IDOR/CSRF checks	**High**
Tampering	Altered votes in Database or forged CSRF token	Parameterized queries + SHA-256 hashing + integrity checks	**Critical**
Repudiation	User denies casting vote	Tamper-evident Audit Logs with SHA-256 hashed entries	**Medium**
Information Disclosure	Leaked votes, session data, or hashed entries from logs	Encrypt sensitive data + secure error handling + restricted logs	**High**
Denial of Service	Overloading Flask App or Dashboard during election	Rate limiting + input validation	**High**
Elevation of Privilege	Bypassing IDOR check to access other voters' data	Strict RBAC + least privilege + proper authorization	**Critical**

3. CVSS Risk Assessment (v3.1)

Each identified threat is scored using the CVSS v3.1 base scoring methodology. Scores were calculated using the NVD CVSS Calculator (https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator).

Threat	CVSS v3.1	Severity	Notes
Spoofing – Session Hijacking / Fake Voter	9.8	Critical	Allows fraudulent votes
Tampering – Vote Manipulation in Database	9.8	Critical	Direct impact on election integrity
Tampering – CSRF / IDOR Bypass	9.5	Critical	Unauthorized vote submission
Information Disclosure – Audit Logs / Errors	8.2	High	Violates voter confidentiality
Repudiation – Weak Audit Logs	6.5	Medium	Voter can deny actions
Denial of Service – App Overload	7.5	High	Disrupts voting process

Scores marked Critical (9.0+) demand immediate remediation. High scores (7.0–8.9) should be addressed before production deployment.

4. Attack Tree

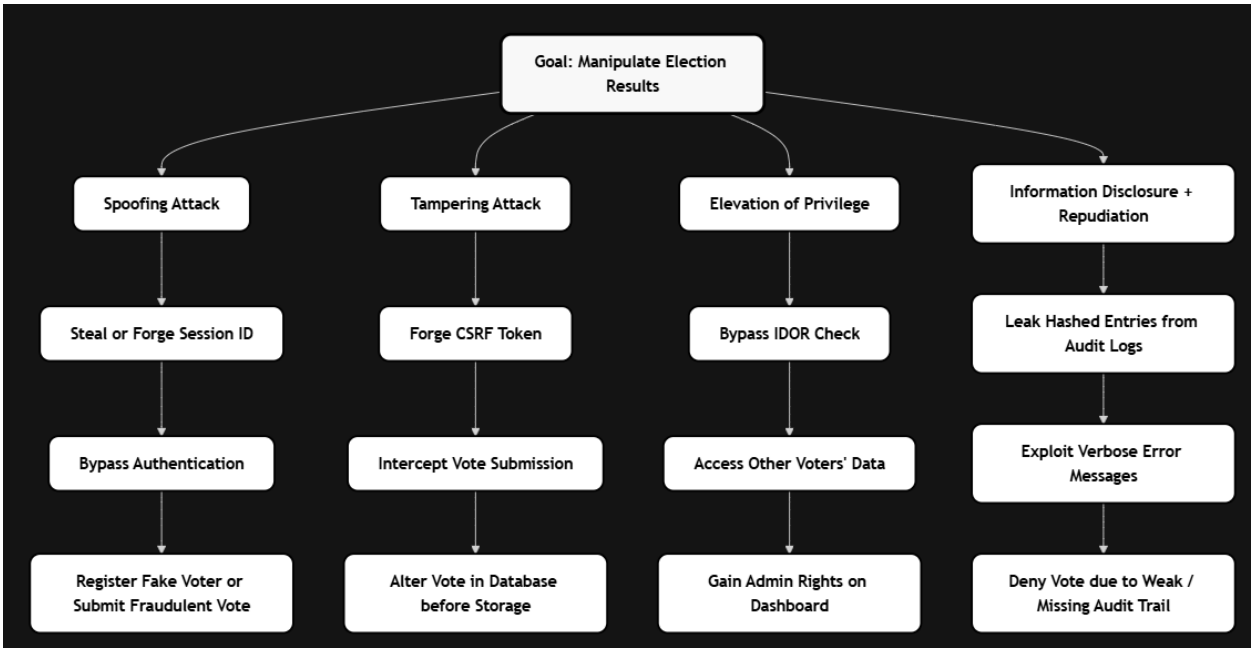
The Attack Tree models the adversary's goal of manipulating election results, decomposing it into distinct attack paths.

Goal: Manipulate Election Results

- Path 1 – Spoofing
  Attacker steals session_id or bypasses authentication → submits fake vote
- Path 2 – Tampering
  Attacker forges CSRF token or tampers with vote before storing in Database
- Path 3 – Elevation of Privilege
  Attacker bypasses IDOR check → accesses unauthorized voter data or Dashboard
- Path 4 – Information Disclosure + Repudiation
  Leaks hashed entries from Audit Logs → denies actions due to weak logging

4.1 Attack Tree Diagram

The Attack Tree for the Online Secure Voting App is given below:



5. Summary and Recommendations

T The Secure Voting System DFD shows clear exposure in the data flows between the Voter, Flask App, Database, Audit Logs, and Dashboard. The Microsoft TMT report identified 103 threats, confirming high risk in authentication, vote submission, and logging.

Most Critical Risks:

- Session hijacking and vote tampering
- Bypassing CSRF/IDOR checks
- Leakage of sensitive data through logs or errors

Priority Mitigations Recommended:

- Enforce CSRF protection and IDOR checks on all vote submissions
- Use parameterized queries and SHA-256 hashing for database operations
- Implement secure session management with short expiry and secure cookies
- Make Audit Logs tamper-evident with cryptographic signing
- Encrypt all sensitive data at rest and in transit (TLS 1.3)
- Add rate limiting and proper input validation
- Conduct regular penetration testing before deployment